The SPSC Lock-Free Ring Buffer

A First-Principles Treatment of Memory Ordering, Cached-Index Optimization, and Unsigned-Overflow Correctness

Robert (Bo) Hou · Robert's Technical Notes · June 2026

Abstract. The single-producer single-consumer (SPSC) lock-free ring buffer is the canonical data backbone of low-latency systems. This report develops the design from first principles in three layers. First, we derive the *minimal* memory ordering required on each atomic operation, establishing a precise criterion for when an *acquire* load is mandatory versus when a *relaxed* load suffices — a distinction routinely over-specified in practice. Second, we analyze the cached-index optimization, proving its safety via a monotonicity argument and clarifying that it reduces *coherence cost*, not correctness or fill rate. Third, we give a formal congruence-theoretic proof that the absolute-counter design remains correct under unsigned hardware overflow, strengthened with an invariant-maintenance induction.

1. The Baseline Design and Notation

We store trivially-copyable elements of type `T` in a fixed array of size `Capacity`, a power of two. Two monotonically increasing absolute counters — `tail_` (written only by the producer) and `head_` (written only by the consumer) — index the buffer through a bitwise mask.

```
template <typename T, size_t Capacity>
class SpscQueue {
    static_assert((Capacity & (Capacity - 1)) == 0,
                  "Capacity must be power of 2");

    alignas(64) std::atomic<size_t> head_{0};  // consumer writes
    alignas(64) std::atomic<size_t> tail_{0};  // producer writes
    alignas(64) std::array<T, Capacity> buf_;

public:
    bool try_push(const T& item) {                        // producer only
        size_t t = tail_.load(std::memory_order_relaxed); // [A]
        size_t h = head_.load(std::memory_order_relaxed); // [B]
        if (t - h == Capacity) return false;              // full
        buf_[t & (Capacity - 1)] = item;
        tail_.store(t + 1, std::memory_order_release);    // [C]
        return true;
    }

    bool try_pop(T& out) {                                // consumer only
        size_t h = head_.load(std::memory_order_relaxed); // [D]
        size_t t = tail_.load(std::memory_order_acquire); // [E]
        if (h == t) return false;                         // empty
        out = buf_[h & (Capacity - 1)];
        head_.store(h + 1, std::memory_order_release);    // [F]
        return true;
    }
};
```

Three structural decisions deserve emphasis up front.

- **Absolute counters, not wrapped indices.** `head_` and `tail_` only ever increase; the physical slot is `counter & (Capacity−1)`. This makes the occupancy `t − h` a single subtraction, with no branch to handle wrap.
- **Power-of-two capacity.** The mask `& (Capacity−1)` is exactly `mod Capacity`, eliminating a division.
- **Per-line alignment.** `head_` and `tail_` occupy separate cache lines (`alignas(64)`) to avoid false sharing: the producer writes only `tail_`, the consumer only `head_`; co-locating them would make the line ping-pong between cores under MESI even though the two variables are logically independent.

2. Memory Ordering: The Minimal Specification

2.1 Two independent facts: atomicity vs. ordering

It is essential to separate two guarantees that `std::atomic` provides.

- **Atomicity**: an atomic load/store is indivisible (no torn reads), and for any single atomic variable all threads observe one consistent *modification order* — even under *relaxed*.
- **Ordering**: whether operations on *different* variables become visible to another thread in a particular order. This is what `memory_order` controls.

A *relaxed* operation gives the first but not the second. This is why reading one's *own* counter is always safe with *relaxed*: by read-your-own-write plus single-variable modification-order consistency, the producer's relaxed load of `tail_` [A] and the consumer's relaxed load of `head_` [D] always observe their own latest store.

2.2 The release/acquire handshake

- **release** (the publisher): on the store that announces "ready," all writes sequenced before it are guaranteed visible to any thread that later reads the announced value. A one-way downward fence — prior writes cannot sink below it.
- **acquire** (the reader): on the load that reads the announced value, all reads sequenced after it cannot be hoisted above it. A one-way upward fence.

The handshake fires *only* when the acquire load actually reads the value written by the release store. At that instant a *synchronizes-with* edge is established, and everything the publisher wrote before its release becomes visible to the reader after its acquire.

> **Remark.** Release/acquire need not be "symmetric." A release store may be paired with a relaxed load and remain legal; the relaxed load simply does not enjoy the synchronization. Whether a load needs acquire depends on what the loading thread does *next*, not on whether the opposite store is release.

2.3 The decisive criterion

> **Proposition (Acquire criterion).** A load of an atomic flag requires acquire ordering *if and only if* the loading thread, after reading the flag, proceeds to read a separate memory region that the publishing thread wrote before its corresponding release store. If the loaded value is used only for arithmetic or a control decision — and no producer-written payload is subsequently read — then relaxed is sufficient.

We apply this to the two cross-thread loads.

[E] — consumer loads `tail_`: must be acquire. After the load, the consumer executes `out = buf_[h]`, reading a slot the producer filled via `buf_[t] = item` before its release store [C]. Crucially, the address `buf_[h]` depends on `h` (the consumer's own counter), *not* on the loaded `t`. Therefore no data dependency forces the payload read to wait for the `tail_` load; only a *control* dependency through `if (h == t)` stands between them, and control dependencies do not prevent speculative execution. A CPU may speculatively execute `out = buf_[h]` before the load resolves, reading stale or partially-written data. The acquire on [E] forbids this reordering; paired with [C]'s release, it guarantees that once the new `t` is observed, `buf_[h]` is already fully written.

[B] — producer loads `head_`: relaxed suffices. After the load, the producer computes `t − h` to test fullness and then *writes* `buf_[t]`. It never *reads* any consumer-written payload (the consumer only consumes; it writes nothing into `buf_` that the producer reads). Hence no acquire is needed: the value of `head_` is consumed purely as an arithmetic quantity.

Remark (Why a stale head cannot corrupt). Suppose [B] is relaxed and the producer reads an outdated `head_`. Because `head_` is monotone, an outdated value is strictly ≤ the true value, so the producer *underestimates* the free space. The worst outcome is a spurious "full" when a slot was in fact available; the producer never mistakenly believes a slot is free when it is not. The error is always biased toward the conservative side, so correctness is preserved. The same reasoning underlies the cached-index optimization of Section 3.

2.4 Summary of the six orderings

Site	Operation	Order	Reason
`[A]`	`tail_.load (own)`	`relaxed`	read-your-own-write
`[B]`	`head_.load (peer)`	`relaxed`	value used for arithmetic only
`[C]`	`tail_.store`	`release`	publishes buf_ payload
`[D]`	`head_.load (own)`	`relaxed`	read-your-own-write
`[E]`	`tail_.load (peer)`	`acquire`	payload buf_[h] read afterward
`[F]`	`head_.store`	`release`	read completes before slot is freed

The single load-bearing handshake is **[C] release ↔ [E] acquire**. Site [F] is release not to pair with [B], but to ensure the consumer's read `out = buf_[h]` completes before it announces the slot as reusable; otherwise the producer might overwrite a slot still being read.

2.5 Platform realization

On x86 (TSO), the entire family of relaxed/acquire/release loads and stores compiles to plain `mov`; the hardware already forbids all reorderings except store→load. Only `seq_cst` stores incur extra cost (an `xchg` or `mov+mfence` to drain the store buffer), which is why low-latency code avoids `seq_cst`. On weakly-ordered architectures (ARM, POWER), acquire/release emit real barrier instructions, so the [B] = relaxed optimization there saves an actual fence. The abstraction is portable: the engineer reasons only about data-and-flag happens-before; the compiler lowers it to each platform's correct instructions.

3. The Cached-Index Optimization

3.1 Motivation: coherence is correct but not free

Every `try_push` that loads `head_` pays a cross-core cost. The consumer has most recently written `head_`, so its cache line is in the Modified state on the consumer's core. When the producer reads it, MESI must transfer ownership across cores — tens to hundreds of cycles — and the subsequent consumer write reclaims the line, producing a ping-pong. MESI guarantees the producer reads the *latest* value, but reading it is expensive. The optimization removes most of these cross-core reads.

3.2 Mechanism

The producer keeps a private, non-atomic `cached_head_`. Most pushes test fullness against this cached value, touching only the producer's own L1. Only when the cached value indicates "full" does the producer perform a real `head_.load()` to refresh.

```
bool try_push(const T& item) {
    size_t t = tail_.load(std::memory_order_relaxed);
    if (t - cached_head_ == Capacity) {              // cheap local test
        cached_head_ = head_.load(std::memory_order_acquire); // refresh
        if (t - cached_head_ == Capacity)
            return false;                            // genuinely full
    }
    buf_[t & (Capacity - 1)] = item;
    tail_.store(t + 1, std::memory_order_release);
    return true;
}
```

3.3 Safety

> **Lemma (Cache monotonicity).** At all times cached_head_ ≤ actual_head, where actual_head is the current value of `head_`.

Proof. `cached_head_` is only ever assigned from a past load of `head_`. Since the consumer increments `head_` monotonically, any previously observed value is ≤ the current one. □

> **Proposition (Conservative correctness).** If the producer writes a slot, that slot is genuinely free; the producer never overwrites unread data.

Proof. The producer writes only when its fullness test fails, i.e. when (after a possible refresh) t − cached_head_ < Capacity. By the monotonicity lemma, t − actual_head ≤ t − cached_head_ < Capacity. Thus the true occupancy is strictly below capacity, so a free slot exists. The cached value can only cause a *spurious full* (a conservative refusal), never a *spurious free*; "false full" is harmless (it triggers one refresh), while "false free" — which would corrupt — cannot occur. □

3.4 What it does *not* do

The optimization does not throttle or control the producer's fill rate, nor does it govern how fast the buffer fills. The fill rate is determined solely by the relative throughput of producer and consumer. The cached index changes only the *cost of the fullness check* — replacing a frequent cross-core load with an occasional one. The producer is ultimately gated by the physical capacity bound, not by the cache. By analogy: the cached index is a cheaper *gauge* for reading the water level; switching gauges does not change how fast water flows in or out.

4. Correctness Under Unsigned Hardware Overflow

This section formalizes why the absolute-counter design remains correct when the k-bit counters overflow and wrap. We first establish the invariant the algorithm maintains, then prove addressing and distance invariance under that invariant.

4.1 Formal setting

Let $\mathbb{Z}_{2^k}$ be the ring of integers modulo 2^k (a k-bit unsigned register, k = 64 typically). Unsigned addition and subtraction are $a \oplus b \equiv a + b \pmod{2^k}$ and $a \ominus b \equiv a - b \pmod{2^k}$. Let $N = 2^m$ with $0 < m < k$ be the capacity, and let the addressing map $f : \mathbb{Z}_{2^k} \to \mathbb{Z}_N$ be the bitwise truncation $f(x) = x \text{ AND } (2^m - 1)$.

> **Lemma (Modular homomorphism of f).** Since the mask $(2^m - 1)$ isolates the low m bits, $f(x) \equiv x \pmod{2^m}$. Because $2^m \mid 2^k$, the map descends through the quotient and preserves additive structure: $f(a \oplus b) \equiv (a+b \bmod 2^k) \bmod 2^m \equiv (a+b) \bmod 2^m \equiv (f(a)+f(b)) \bmod 2^m$.

4.2 The maintained invariant

Let $t, h \in \mathbb{Z}_{2^k}$ be the producer and consumer absolute counters.

> **Invariant (No-overflow occupancy).** At every reachable state, $0 \le (t \ominus h) < N$.

All subsequent correctness rests on this invariant, so we prove the algorithm maintains it, rather than assuming it.

> **Theorem 1 (Invariant maintenance).** Under single-producer single-consumer operation, the invariant $0 \le t \ominus h < N$ holds in every reachable state.

Proof. By induction over operations.

Base case. Initially $t = h = 0$, so $t \ominus h = 0$, and $0 \le 0 < N$.

Inductive step. Assume $0 \le t \ominus h < N$. Two operations can change the state.

Push. The producer advances $t \mapsto t \oplus 1$ only after its guard verifies $t \ominus h < N$, i.e. $t \ominus h \le N-1$. After the increment the occupancy becomes $(t \oplus 1) \ominus h = (t \ominus h) + 1 \le (N-1) + 1 = N$. Combined with the lower bound, the new occupancy lies in $[1, N]$, within the admissible

range. (At the extremal value N the buffer is exactly full, and the next push guard correctly rejects.) Only the producer modifies t, so h is unchanged and the lower bound $0 \le t \ominus h$ is preserved.

Pop. The consumer advances $h \mapsto h \oplus 1$ only after verifying $t \ominus h > 0$ (non-empty). After the increment, $t \ominus (h \oplus 1) = (t \ominus h) - 1 \ge 0$, preserving the lower bound; the upper bound is preserved since occupancy strictly decreases. Only the consumer modifies h, so t is unchanged.

Because the producer modifies only t and the consumer only h, the two updates never race on the same counter, and each preserves both bounds. Hence the invariant holds in all reachable states. □

4.3 Addressing invariance

> **Theorem 2 (Seamless addressing across overflow).** When the producer counter $t_1 = 2^k - 1$ increments to $t_2 = t_1 \oplus 1 = 0$, the physical index remains sequential: $f(t_2) = (f(t_1) + 1) \bmod N$.

Proof. **Left side.** $t_2 = 0$, so $f(t_2) = f(0) \equiv 0 \pmod{2^m}$, and since $0 \in [0, 2^m - 1]$, $f(t_2) = 0$.

Right side. $(f(t_1) + 1) \bmod 2^m \equiv (t_1 + 1) \bmod 2^m \equiv (2^k - 1 + 1) \bmod 2^m \equiv 2^k \bmod 2^m$. Because $2^m \mid 2^k$, this is 0.

Both sides equal 0, so by transitivity $f(t_2) = (f(t_1) + 1) \bmod N = 0$. The wrap of the absolute counter produces no discontinuity in the physical slot sequence. □

4.4 Distance invariance

> **Theorem 3 (Distance invariance via ring arithmetic).** Suppose the producer has wrapped so its nominal value is below the consumer's: $t_2 = 0 < h$, with $h = 2^k - d$ for the true physical distance $0 < d < N$. Then the unsigned subtraction yields the exact distance with no branch: $t_2 \ominus h = d$.

Proof. In $\mathbb{Z}_{2^k}$, subtraction is addition of the additive inverse: $t_2 \ominus h \equiv (t_2 + 2^k - h) \bmod 2^k = (0 + 2^k - (2^k - d)) \bmod 2^k = d \bmod 2^k$. By the maintained invariant, $0 < d < N < 2^k$, so d lies in the fundamental domain and the outer reduction is the identity: $d \bmod 2^k = d$. Hence $t_2 \ominus h = d$. □

> **Remark (Two's complement is the implementation, not the definition).** Because t, h are unsigned, the operative algebra is ring arithmetic modulo 2^k. The term $2^k - h$ is the additive inverse of h in this ring; its bit pattern happens to coincide with the two's-complement encoding (`NOT h + 1`), which is why a single unsigned subtraction on the ALU produces the correct distance even when $t < h$ numerically. The hardware corrects the apparent "temporal inversion" for free.

4.5 Worked bitstream trace

Let k = 8 ($2^k = 256$) and N = 8. Suppose h = 250 (`1111 1010`) and the producer wraps to t = 0 (`0000 0000`); the true distance is 256 − 250 = 6.

- **Additive inverse of h:** NOT `1111 1010` + 1 = `0000 0101` + 1 = `0000 0110` (value 6).
- **Unsigned add** t + (2^k − h): `0000 0000` + `0000 0110` = `0000 0110` (value 6).
- **Mask** & (N−1) = & `0000 0111`: `0000 0110` AND `0000 0111` = `0000 0110` (slot 6).

The cyclic topology of modular arithmetic absorbs the wrap with zero branches and zero misprediction penalty.

5. Synthesis

The SPSC ring buffer rewards reasoning at the right level of abstraction. Correctness of *ordering* reduces to a single question — does the loading thread subsequently read payload the other thread published? — which dictates acquire versus relaxed without invoking store buffers or MESI at the point of use. Correctness of the *cached index* reduces to monotonicity: a stale peer counter only ever errs conservatively. Correctness of *addressing* reduces to two modular facts: the mask is a homomorphism, and unsigned subtraction is ring subtraction. The deep hardware mechanisms — store buffers, cache coherence, speculative execution — explain *why* these abstractions are sound, but the working engineer reasons in terms of data-and-flag happens-before, monotone counters, and congruences. That is the level at which the code is both written and defended.